

Tries (Prefix Trees)

A derivation-first interview study guide

Tries (Prefix Trees) — Study Guide

Roadmap

This guide follows the same arc the source lesson used to derive Tries from first principles, then layers on the interview patterns built on top of the core structure.

1. **The problem a Trie solves** — why shared prefixes matter and why a HashSet isn't enough
2. **The Trie node** — `children` and `is_end`, and the distinction between them
3. **Core operations** — `insert`, `search`, `startswith`, derived step by step, including a real correction worked through live in the source chat
4. **Interview patterns**, in the order the source chat built them: - Autocomplete (Trie + DFS) - Wildcard search (Trie + DFS/backtracking) — *Add and Search Word* - Prefix pruning in a grid (Trie + backtracking) — *Word Search II* - Shortest-prefix replacement — *Replace Words* - Augmented Trie nodes — *Map Sum Pairs* - Binary Trie — *Maximum XOR of Two Numbers in an Array* - Prefix-gated DFS — *Longest Word in Dictionary* - Trie + DP — *Word Break*
5. **Two canonical extensions** the task list calls out as typical Trie patterns worth knowing even though the source chat only named them in passing: *Prefix and Suffix Search* and *Design Search Autocomplete System*, plus *Implement Trie II* as a compact recap of "augment the node with more than one counter."
6. **Recognition guide, decision table, and interview Q&A drills**

Every code block in this guide is pulled directly from `sol.py`, which is tested against independent brute-force implementations in `test_sol.py` (hundreds of random trials per function). Every trace/step-table is asserted against the same `sol.py` before being rendered here — nothing in this guide is hand-typed pseudocode that might silently drift from the real implementation.

1. The problem a Trie solves

Suppose we store:

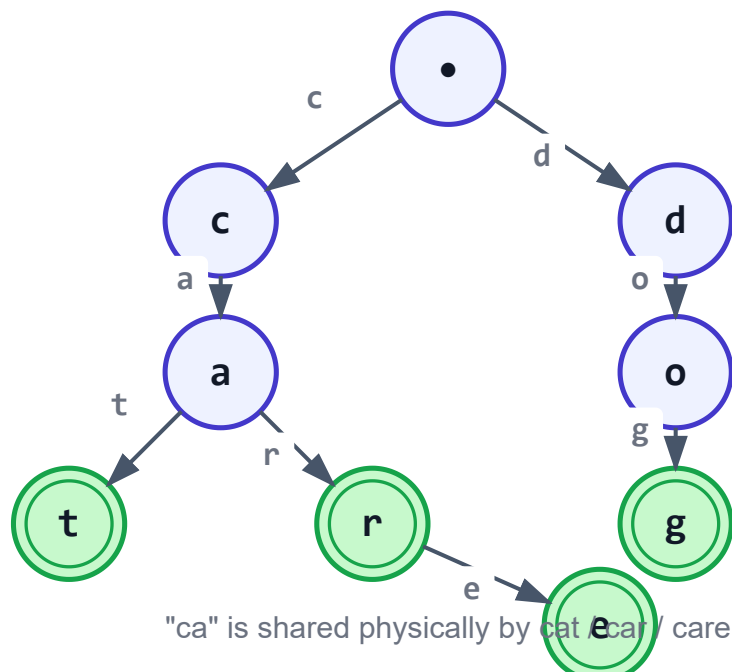
```
cat
car
care
dog
```

We want to answer questions like:

- Does `"care"` exist?
- Does any word start with `"ca"` ?
- What words can follow `"app"` ?
- Is `"car..."` still a valid beginning of some dictionary word?

The key observation: **many words share the same beginning.** `cat` , `car` , and `care` all start with `c → a` . A Trie makes that shared structure *physically explicit* — instead of three separate strings sitting side by side in memory, the shared prefix `"ca"` is stored exactly once, and the words fan out from wherever they first diverge.

Trie for {cat, car, care, dog}



Vocabulary

Term	Meaning
root	The starting point. It doesn't represent a character — it represents the empty string.
edge	A connection between two nodes; it represents one character transition.
node	A position in a character sequence — i.e., a prefix seen so far.
path	A path from the root spells out a string/prefix. <code>root → c → a</code> spells <code>"ca"</code> .
prefix	The beginning of a string. <code>""</code> , <code>"c"</code> , <code>"ca"</code> , <code>"car"</code> , <code>"care"</code> are all prefixes of <code>"care"</code> .
complete word / terminal marker	Whether a <i>stored word</i> ends at this exact node — this is <code>is_end</code> .

Core mental model: A Trie is a tree where paths represent sequences, and shared paths represent shared prefixes.

2. Why not just a HashSet?

A `HashSet` like `{"cat", "car", "care", "dog"}` is excellent for one question: *"does this exact key exist?"* It answers `"car"` in words in $O(1)$ expected time.

But it is organized around **complete keys**, not their internal character structure. Ask it *"does any stored word start with `"ca"`?"* and there is no shortcut — you must scan every word and call `.startswith("ca")`, because the hash of `"car"` and the hash of `"care"` have nothing to do with each other; the `HashSet` threw away the fact that they share a prefix.

A Trie makes the prefix structure the *primary* thing it's organized around. Walking `root → c → a` directly locates the `"ca"` prefix — no scanning required.

Need	Natural choice	Why
Exact word membership	HashSet	Simple exact lookup
Key → value	HashMap	Maps keys to values
Does prefix exist?	Trie	Prefix is a path, not a key
Autocomplete	Trie	Find prefix node, then DFS
Wildcard dictionary search	Trie + DFS	Multiple possible branches
Prefix pruning in backtracking	Trie + DFS	Quickly reject invalid prefixes

Caveat: a Trie is not automatically better. It can use significantly more memory than a `HashSet`, since every character transition may need its own node and child references. The interview question to ask yourself is: *"do I actually need the structure of the strings, or only exact membership?"* If only exact membership: `HashSet`. If prefix structure itself matters: Trie.

3. The Trie node

Conceptually, every node needs exactly two things:

```
TrieNode
├─ children  -- what characters can come next?
└─ is_end   -- does a complete word end here?
```

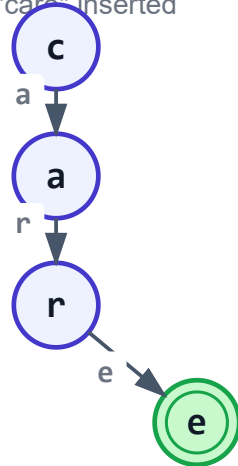
`children` maps a character to the next `TrieNode`. For our example words (`cat`, `car`, `care`), the node representing `"ca"` has two children: `'t'` and `'r'`.

The critical distinction

Suppose we've inserted only `"care"`. The path `root → c → a → r` exists (it's on the way to `"care"`), but `"car"` itself was **never inserted**.

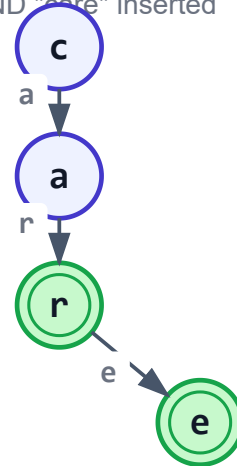
The is_end distinction: prefix vs. stored word

only "care" inserted



r.is_end=False

"car" AND "care" inserted



r.is_end=True

The same node/path can therefore represent **both** a prefix *and* a complete word, depending only on whether `is_end` was set there:

```
node("ca").is_end = False    # "ca" was never inserted
node("car").is_end = True     # only if "car" was itself inserted
node("care").is_end = True    # "care" was inserted
```

So:

- `children` tells us what can come next.
- `is_end` tells us whether a word ends *here*, at this exact node.

That is the entire conceptual node:

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False
```

4. Insertion — derived step by step

Suppose the Trie is empty and we insert "care". We process one character at a time, starting `current = root`, and for each character we ask: **does current already have a child for this character?**

- **No** → create it, then move into it.
- **Yes** → just move into it (reuse).

After the last character, mark `current.is_end = True`.

```

start at root
|
for every character
|
does child exist?
/      \
yes      no
|        |
move     create
|        |
└─→ move
      |
    end of word
      |
    is_end = True

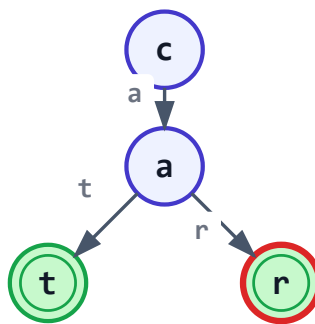
```

Dry run: insert("cat"), then insert("car"), then insert("care")

insert("cat") -> insert("car") -> insert("care")



cat



car (r is new, red = just created)

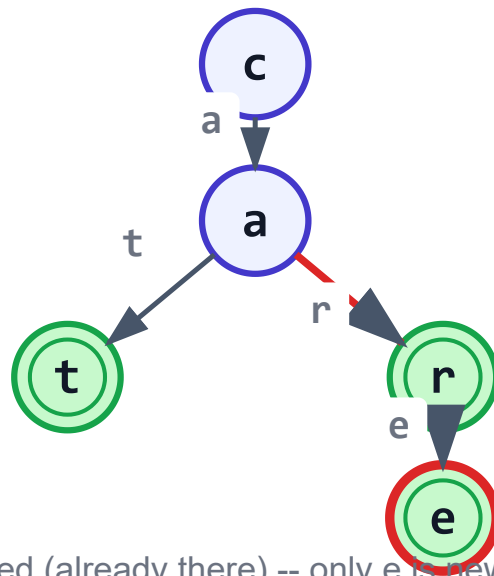
"cat" creates 3 brand-new nodes (c , a , t) since the Trie started empty. Inserting "car" next reuses c and a (they already exist from "cat") and creates only one new node, r .

insert(word)	result	note
bat	inserted	root now has children: b
ball	inserted	root now has children: b

Reusing an existing path entirely

Now suppose "care" is already fully inserted, and we insert "car" . Every character c , a , r already exists on the path to "care" — we don't create a second c-a-r chain. We simply walk the existing path and, upon reaching the r node, set is_end = True there too.

insert("care") after cat, car already exist



c,a,r reused (already there) -- only e is newly created

Before inserting "car": `node("car").is_end = False`, `node("care").is_end = True`. After: **both** are `True`. This is exactly why a Trie saves space over storing "car" and "care" as two independent strings — the shared `c → a → r` prefix physically exists only once.

```
def insert(self, word):
    cur = self.root
    for ch in word:
        if ch not in cur.children:
            cur.children[ch] = TrieNode()
        cur = cur.children[ch]
    cur.is_end = True
```

Complexity. Let `L` = length of the word. Insertion visits exactly one node per character: **$O(L)$** time, **$O(L)$** additional space in the worst case (if none of the path already existed). Across an entire Trie, total space is proportional to the number of *unique* character positions after prefix sharing — not the sum of every word's raw length.

5. Exact search — why reaching the node isn't enough

Suppose only "ball" was inserted, and we ask `search("ba1")`. We can successfully walk `root → b → a → 1` — every character exists. But should we return `True`?

No. The path existing only tells us "ba1" is a *prefix* of something stored. It does not tell us "ba1" was itself inserted as a complete word. We must additionally check `is_end` at the final node:

```
def search(self, word):
    node = self._walk(word)
    return node is not None and node.is_end
```

If any required child is missing partway through, we can return `False` immediately — no need to keep walking.

Dry run

op	arg	result
search	car	True
search	ca	False
search	cab	False
startsWith	ca	True
startsWith	car	True
startsWith	cab	False
startsWith	do	True

Complexity. For `L` = length of the queried word: **$O(L)$** time, **$O(1)$** extra space (the Trie itself occupies the space discussed above).

6. Prefix search — `startsWith`

This is the operation that gives a Trie its name ("prefix tree"). The only difference from `search` is what we check at the end:

```
search:      follow path -> if missing, False -> return is_end
startsWith: follow path -> if missing, False -> return True
```

`startsWith` **never checks** `is_end` — it only cares whether the path exists, because a prefix doesn't need to have been inserted as a complete word to "exist" as a prefix.

```
def startswith(self, prefix):
    return self._walk(prefix) is not None
```

A correction worked through live in the source chat

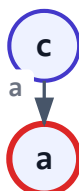
The source lesson asked: given a Trie built from `cat`, `car`, `care`, does checking whether the **prefix** `"ca"` exists require checking `is_end`?

The first answer given was **A) Yes**, reasoned from "only words ending with t, r, e were inserted" — i.e. reasoning about which letters the stored words happen to end with. That reasoning doesn't actually answer the question being asked. The correct answer is **B) No** — and the *reason* matters more than the letter: `search(word)` and `startsWith(prefix)` are two **different questions** about the same node, and only one of them needs `is_end`.

Correction: does checking a PREFIX need `is_end`?

Trie built from cat, car, care -- query the node for "ca"

search("ca") -- exact word



node("ca") exists, `is_end=False`

needs path + `is_end`

-> **False** (`is_end` is False)

startsWith("ca") -- prefix



node("ca") exists, `is_end=False`

needs path only

-> **True** (no `is_end` check)

user's first answer ("A") reasoned from which words end in t/r/e --
the real rule is about WHICH QUESTION is asked, not which letters exist

call	rule	result
<code>search("ca")</code>	needs path + <code>is_end</code>	False
<code>startsWith("ca")</code>	needs path only	True

Exact word → path exists AND `is_end` . **Prefix** → path exists. (`is_end` is irrelevant.)

This distinction — *which question is being asked, not which letters happen to be stored* — is the fundamental Trie rule that everything else in this guide builds on. A concrete illustration from the same lesson: if only `apple` and `application` are stored, then `search("app") == False` (no word `"app"` was ever inserted) but `startsWith("app") == True` (the path exists, since both words pass through it).

Complexity. Same as `search` : **O(L)** time, **O(1)** extra space.

7. The complete core Trie

```

class TrieNode:
    __slots__ = ("children", "is_end", "word")

    def __init__(self):
        self.children = {}
        self.is_end = False
        self.word = None
  
```

```

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        cur = self.root
        for ch in word:
            if ch not in cur.children:
                cur.children[ch] = TrieNode()
            cur = cur.children[ch]
        cur.is_end = True

    def search(self, word):
        node = self._walk(word)
        return node is not None and node.is_end

    def startsWith(self, prefix):
        return self._walk(prefix) is not None

    def _walk(self, s):
        cur = self.root
        for ch in s:
            if ch not in cur.children:
                return None
            cur = cur.children[ch]
        return cur

```

Three questions summarize the whole core:

INSERT	"Do I have this character?"	no->create, yes->reuse; finally is_end=True
SEARCH	"Can I follow every character?"	no->False; yes->return is_end
STARTSWITH	"Can I follow every character?"	no->False; yes->return True

All three run in **O(L)** where **L** is the word/prefix length — never $O(1)$, and never proportional to the number of stored words (that's the whole point of the structure).

Pattern 1 — Autocomplete (Trie + DFS)

Recognition

"Given a prefix, return all matching words." Any variant of "starts with", "suggest completions", or "what can follow this prefix" is this pattern.

Thought process

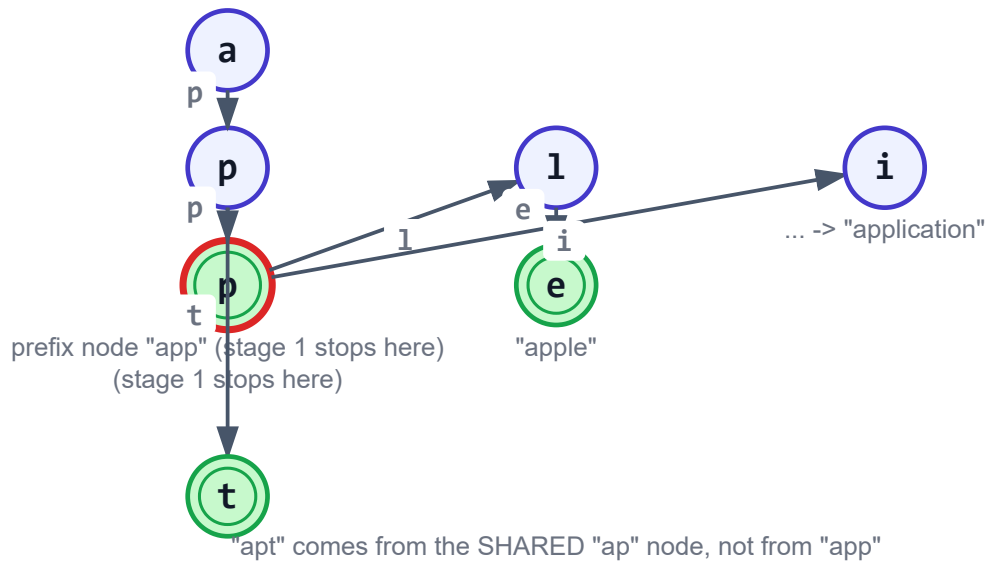
Autocomplete splits cleanly into two independent stages:

1. Walk to the prefix node (ordinary Trie traversal, $O(P)$)
2. DFS/BFS downward from there (collect every `is_end==True` node)

The Trie does **not** magically hand you the list of completions — it only gives you the correct *starting point*. From there, DFS explores every descendant, and whenever `node.is_end == True` you've found a complete stored word.

Autocomplete("app") on {app, apple, application, apt, banana}

Stage 1: walk to the prefix node Stage 2: DFS collects is_end nodes



Notice in the figure that `"apt"` is collected via the **shared** `"ap"` node, not from `"app"` — DFS explores every branch under the prefix node, and `"app"` itself is one of the branches (`is_end=True` right at the prefix node counts too, since a prefix can itself be a complete word).

Code

```
def autocomplete(root, prefix):
    """Given a Trie root and a prefix, return every stored word that starts
    with that prefix, in the order DFS discovers them (sorted for a stable
    test/trace result).
    """
    cur = root
    for ch in prefix:
        if ch not in cur.children:
            return []
        cur = cur.children[ch]

    result = []

    def dfs(node, path):
        if node.is_end:
            result.append(prefix + "".join(path))
        for ch, child in sorted(node.children.items()):
            path.append(ch)
            dfs(child, path)
            path.pop()

    dfs(cur, [])
    return result
```

field	value
prefix	"app"
matches	app, apple, application

Complexity

Let P = prefix length, K = total characters across the words actually returned/explored in the subtree.

Finding the prefix node: $O(P)$. The DFS: $O(K)$. Total: $O(P + K)$ — proportional to the *output*, not to the total dictionary size.

Pattern 2 — Wildcard Search (Trie + DFS/backtracking)

Recognition

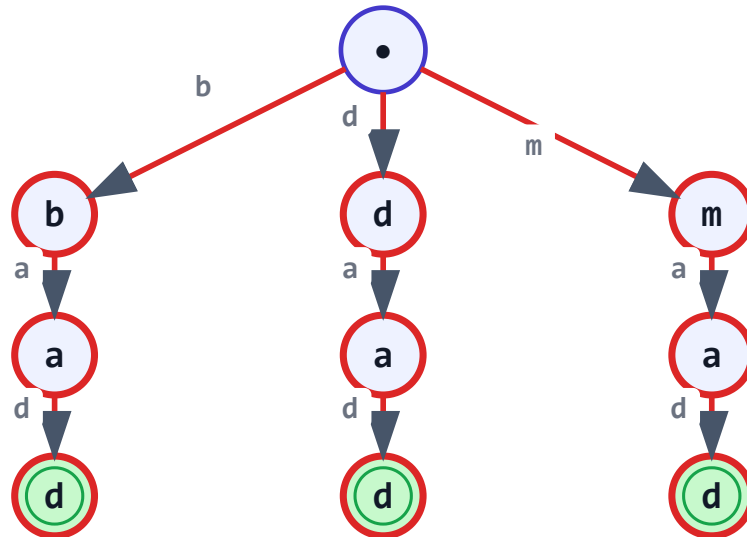
Design Add and Search Words Data Structure — **LC 211**. Any problem where a query character can mean "any character" (or any of several options).

Thought process

Ordinary Trie traversal follows exactly one child per character. A wildcard breaks that assumption: at a wildcard position, we don't know which branch is correct, so we must **try every existing child** and recurse. This is the jump from plain Trie traversal to **Trie + DFS**:

normal character -> follow exactly one child
wildcard '.' -> try every existing child, DFS each possibility

search(".ad") on {bad, dad, mad} -- wildcard tries EVERY child



'.' at index 0 forks into 3 DFS branches; each then matches 'a','d' normally

Code

```
class WordDictionary:
    def __init__(self):
        self.root = TrieNode()

    def addWord(self, word):
        cur = self.root
        for ch in word:
            if ch not in cur.children:
                cur.children[ch] = TrieNode()
            cur = cur.children[ch]
        cur.is_end = True

    def search(self, word):
        def dfs(node, i):
            if i == len(word):
                return node.is_end
            ch = word[i]
            if ch != '.':
                nxt = node.children.get(ch)
                return nxt is not None and dfs(nxt, i + 1)
            # wildcard: try every existing child
            for child in node.children.values():
                if dfs(child, i + 1):
                    return True
            return False

        return dfs(self.root, 0)
```

pattern	search() result
.ad	True
b..	True
...	True
....	False

Complexity

For a pattern with no wildcards: **$O(L)$** , same as ordinary search. With wildcards, branching can occur at every wildcard position; worst case (a pattern of all dots against a full 26-ary alphabet) is roughly **$O(26^L)$** .

Recursion depth (extra space) is **$O(L)$** .

The Trie handles the prefix structure; DFS handles the multiple possible paths the wildcard introduces. Neither piece alone solves the problem.

Pattern 3 — Prefix Pruning in a Grid (Trie + Backtracking)

Recognition

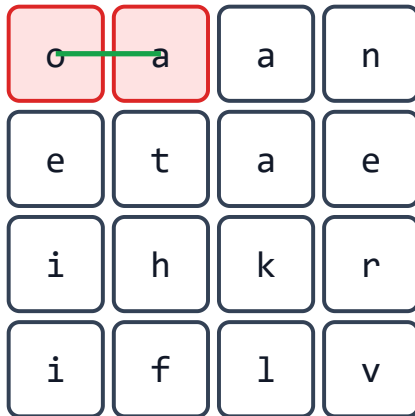
Word Search II — LC 212. A grid/board plus a dictionary of words to find, where a naive backtracking DFS over the board would repeatedly ask *"is this partial string still a valid prefix of some dictionary word?"*

Thought process

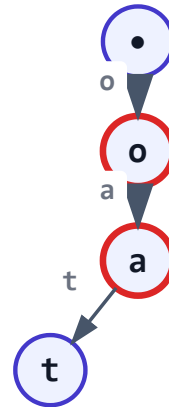
Without a Trie, board DFS might build "o", "oa", "oat", "oath", ... one character at a time, and at every step you'd need some way to check *"could this possibly still become a dictionary word?"* — normally that means scanning the whole dictionary with `.startswith()`, over and over.

A Trie answers that question **incrementally, in $O(1)$ per character**: walk the same path in the Trie that the board DFS is walking on the grid. If the Trie has no child for the next character, **no dictionary word can possibly continue this way** — prune the branch immediately, without touching the dictionary at all.

Board DFS



o -> a walks the SAME path in the Trie



board's next cell is 'a' again (o,a,a)

but Trie node("oa") has no 'a' child

-> prune this branch immediately, don't recurse

Backtracking	generates candidate strings
+	
Trie	kills impossible candidates the instant they stop being a valid prefix
=	
Trie + backtracking	

Code

```
def findWords(board, words):
    root = TrieNode()
    for word in words:
        cur = root
        for ch in word:
            if ch not in cur.children:
                cur.children[ch] = TrieNode()
            cur = cur.children[ch]
        cur.word = word          # stash the full word at the terminal node
        cur.is_end = True

    rows, cols = len(board), len(board[0])
    result = []

    def dfs(r, c, node):
        ch = board[r][c]
        nxt = node.children.get(ch)
        if nxt is None:
            return
        if nxt.is_end:
            result.append(nxt.word)
            nxt.is_end = False    # avoid duplicate collection

        board[r][c] = '#'
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and board[nr][nc] != '#':
                dfs(nr, nc, nxt)
        board[r][c] = ch

    # prune: a leaf with no children is dead weight for future searches
    if not nxt.children:
        del node.children[ch]

    for r in range(rows):
        for c in range(cols):
            dfs(r, c, root)
    return result
```

A small but important detail in the implementation above: once a word is collected, its terminal node's `is_end` is reset to avoid re-adding the same word from a different board path, and leaf nodes with no remaining children are pruned from their parent's `children` dict so later DFS calls from other board cells don't waste time re-checking dead branches.

dictionary word	outcome
oath	found
pea	not found -- pruned or absent
eat	found
rain	not found -- pruned or absent

Complexity

Let N = number of board cells, L = max word length. Building the Trie is $O(\text{sum of word lengths})$. The board DFS is bounded by $O(N * 4^L)$ in the worst case without pruning, but the Trie's incremental prefix check prunes the vast majority of branches in practice — this is the entire reason Word Search II is tractable with a Trie and painfully slow with a plain `HashSet` of words.

Pattern 4 — Shortest-Prefix Replacement

Recognition

Replace Words — LC 648. "Replace each word in a sentence with the shortest dictionary word that is a prefix of it."

Thought process

For each sentence word, walk it through the Trie one character at a time. The **first** time we hit `is_end == True`, we've found the shortest dictionary word that is a prefix of the current word — because we walk left to right, the first terminal node we reach is necessarily the *shortest* matching prefix. Stop immediately; don't keep walking.

replaceWords: walk left-to-right, stop at FIRST is_end



word: "cattle"

c -> a -> t is_end=True here
STOP: don't keep walking t-l-e

result root = "cat"

"cattle" -> "cat"

Code

```
def replaceWords(dictionary, sentence):
    root = TrieNode()
    for word in dictionary:
        cur = root
        for ch in word:
            if ch not in cur.children:
                cur.children[ch] = TrieNode()
            cur = cur.children[ch]
        cur.is_end = True

    def find_root(word):
        cur = root
        prefix = []
        for ch in word:
            if ch not in cur.children:
                return word
            cur = cur.children[ch]
            prefix.append(ch)
            if cur.is_end:
                return "".join(prefix)
        return word

    return " ".join(find_root(w) for w in sentence.split())
```

original word	replaced with
the	the
cattle	cat
was	was
rattled	rat
by	by
the	the
battery	bat

Complexity

Building the dictionary Trie: $O(\text{sum of dictionary word lengths})$. Replacing: $O(\text{sentence length})$ total, since each sentence word is walked at most once, character by character.

Pattern 5 — Augmented Trie Nodes: Map Sum Pairs

Recognition

Map Sum Pairs — LC 677. `insert(key, val)` then `sum(prefix)` returns the sum of values of every inserted key that starts with `prefix`.

Thought process

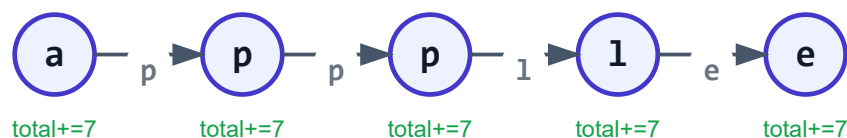
This is the first pattern where the Trie node stores more than `children / is_end` — it's a general technique: **a Trie node can be augmented with whatever extra bookkeeping the problem needs**, updated as you insert.

A naive approach: walk to `node(prefix)`, then DFS the whole subtree summing every `is_end` node's value — this works but re-does the DFS on every `sum()` call. The optimized approach stores a **running total** at every node on a key's path, so `sum(prefix)` becomes a single $O(\text{len})$ walk with no DFS at all.

The subtlety: what if a key is inserted **twice** with different values (`apple=3` , later `apple=10`)? We must not simply add `10` on top of the old `3` — that would double count. Instead we track each key's last value and add only the **delta** ($10 - 3 = 7$) to every node on its path.

MapSum: nodes carry a running total, updated by delta

`insert("apple", 3)` then `insert("apple", 10)` -- delta = $10 - 3 = 7$



`sum("ap")` reads `node("ap").total` directly -- $O(\text{len}(\text{prefix}))$, no DFS needed

Code

```
class MapSumNode:
    __slots__ = ("children", "total")

    def __init__(self):
        self.children = {}
        self.total = 0
```

```

class MapSum:
    def __init__(self):
        self.root = MapSumNode()
        self.values = {}

    def insert(self, key, val):
        delta = val - self.values.get(key, 0)
        self.values[key] = val
        cur = self.root
        cur.total += delta
        for ch in key:
            if ch not in cur.children:
                cur.children[ch] = MapSumNode()
            cur = cur.children[ch]
            cur.total += delta

    def sum(self, prefix):
        cur = self.root
        for ch in prefix:
            if ch not in cur.children:
                return 0
            cur = cur.children[ch]
        return cur.total

```

operation	effect
insert("apple", 3)	sum("ap")=3
insert("app", 2)	sum("ap")=5
insert("apple", 10) (delta=+7)	sum("ap")=12

Complexity

insert : $O(L)$ where $L = \text{len}(\text{key})$. sum : $O(P)$ where $P = \text{len}(\text{prefix})$ — no DFS needed, since the total is already precomputed at every node.

Pattern 6 — Binary Trie: Maximum XOR of Two Numbers in an Array

Recognition

LC 421. "Find $\max(\text{nums}[i] \text{ XOR } \text{nums}[j])$ over all pairs." This is the pattern that reveals a deeper truth about Tries:

A Trie is not fundamentally about letters. It's fundamentally about navigating a branching sequence of choices. The choices can be characters — or they can be bits.

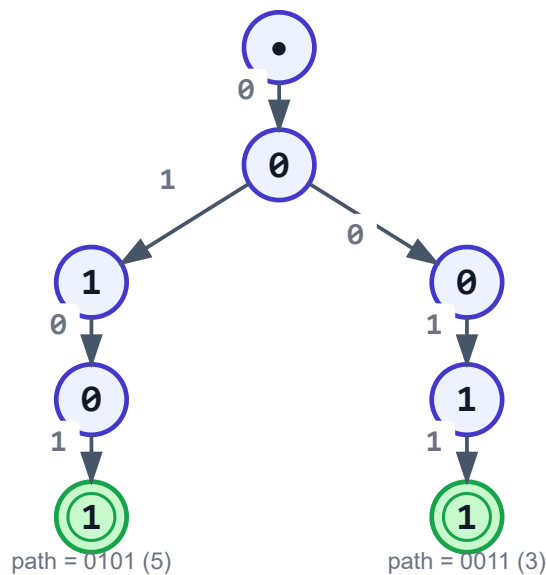
Thought process

XOR wants bits to *differ*: $0 \text{ XOR } 1 = 1$ and $1 \text{ XOR } 0 = 1$, while $0 \text{ XOR } 0 = 0$ and $1 \text{ XOR } 1 = 0$. So to maximize XOR, at every bit position we want to pair a number with something whose bit is the **opposite** of its own.

That "at every position, prefer the opposite choice" structure is exactly what a **binary Trie** represents: insert every number as a fixed-width sequence of bits (most-significant bit first), building a Trie with at most 2 children per node (0 and 1) instead of up to 26.

Binary Trie for Maximum XOR (nums = 5, 3 as 4-bit examples)

5 = 0101 3 = 0011 -- Trie stores bits MSB-first



Then, for each number, greedily walk the Trie: at every bit, if the *opposite* bit exists as a child, take it (this contributes a 1 bit to the XOR result); otherwise you're forced to take the same bit (contributes a 0).

Querying 5 (0101): at each bit, prefer the OPPOSITE child

num bit=0 want 1 opposite exists	num bit=1 want 0 opposite exists	num bit=0 want 1 opposite missing	num bit=1 want 0 opposite missing
--	--	---	---

when the opposite bit exists, take it (XOR bit = 1); else fall back
maximum XOR for 5 against {3} found greedily, one bit at a time

Code

```
class BitNode:
    __slots__ = ("children",)

    def __init__(self):
        self.children = {}

def findMaximumXOR(nums, bits=31):
    root = BitNode()
    for num in nums:
        cur = root
        for bit in range(bits, -1, -1):
            b = (num >> bit) & 1
            if b not in cur.children:
                cur.children[b] = BitNode()
            cur = cur.children[b]

    best = 0
    for num in nums:
        cur = root
        value = 0
        for bit in range(bits, -1, -1):
            b = (num >> bit) & 1
            want = 1 - b
            if want in cur.children:
                value |= (1 << bit)
                cur = cur.children[want]
            else:
                cur = cur.children[b]
        best = max(best, value)
    return best
```

field	value
nums	[3, 10, 5, 25, 2, 8]
max XOR pair	$5 \wedge 25 = 28$
findMaximumXOR(nums)	28

Complexity

Let N = array length, B = bit width (32 for standard 32-bit integers). Building the Trie: $O(N \cdot B)$. Querying every number greedily: $O(N \cdot B)$. Total: $O(N \cdot B) = O(N)$ for fixed-width integers — far better than the naive $O(N^2)$ all-pairs comparison.

This is intentionally kept as an advanced extension rather than part of the core lesson — the important takeaway is the *generalization*, not memorizing this specific problem: whenever a problem involves bit-by-bit greedy decisions over fixed-width numbers, consider a binary Trie.

Pattern 7 — Prefix-Gated DFS: Longest Word in Dictionary

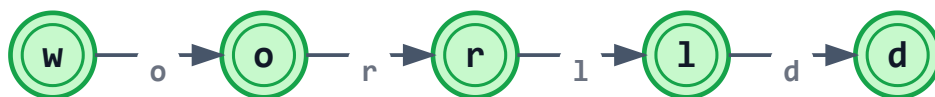
Recognition

LC 720. "Find the longest word such that every prefix of it is also a stored word." For example, given {w, wo, wor, worl, world}, "world" qualifies because w, wo, wor, worl, and world are **all** individually stored words.

Thought process

Build the Trie as usual, then DFS from the root — but with one crucial restriction: **only step into a child if that child's is_end is already True**. If a node is reached but isn't itself `is_end`, its entire subtree is unreachable by this DFS, no matter how many words are technically stored further down — because continuing past a non-terminal node would mean some prefix along the way was never itself a complete word.

longestWord: DFS may only step through an `is_end` child



every prefix w/wo/wor/worl/world is itself a stored word -> keep walking



"bana" has no is_end -> DFS stops before "banana" is reached

Code

```
def longestWord(words):
    root = TrieNode()
    for word in words:
        cur = root
        for ch in word:
            if ch not in cur.children:
                cur.children[ch] = TrieNode()
            cur = cur.children[ch]
        cur.is_end = True

    best = ""

    def dfs(node, path):
        nonlocal best
        word = "".join(path)
        if len(word) > len(best) or (len(word) == len(best) and word < best):
            best = word
        for ch, child in node.children.items():
            if child.is_end:
                # only walk through *complete* prefixes
                path.append(ch)
                dfs(child, path)
                path.pop()

    dfs(root, [])
    return best
```

candidate	reason	outcome
world	w,wo,wor,world all stored	kept
banana	"banan" never inserted	rejected -- DFS never reaches it

Complexity

Building the Trie: $O(\text{sum of word lengths})$. The DFS visits each node at most once: **$O(\text{sum of word lengths})$** total. Note this is a different kind of "prune" from Word Search II's prefix pruning — here we prune based on `is_end`, not based on whether a *path exists at all*.

Pattern 8 — Trie + DP: Word Break

Recognition

Segmenting a string using a dictionary (**LC 139 / 140**-shaped: "can `s` be segmented into dictionary words?"), where the dictionary is walked with a Trie instead of re-slicing the string and hashing every substring.

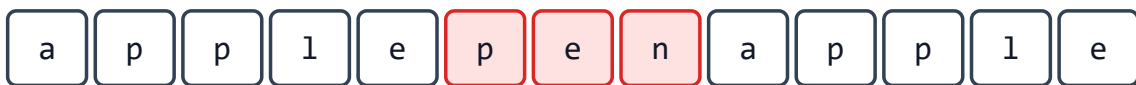
Thought process

At every starting index `i`, walk the Trie one character at a time through `s[i]`, `s[i+1]`, `...`. Whenever the current Trie node has `is_end == True`, a dictionary word ends at this point — check whether the **remainder** of the string (from the next index onward) can also be segmented. This "can the rest be segmented" question is a classic DP subproblem, so this pattern is **Trie + DP**: the Trie finds candidate word boundaries in a single incremental walk per start index, instead of re-testing every possible substring length against a HashSet.

s = "applepenapple", words = {apple, pen}



from i=0: walk Trie 'a','p','p','l','e' -> is_end, dp[5]?



from i=5: walk 'p','e','n' -> is_end, dp[8]? recurse on suffix

dp[i] = True if some dictionary word starts at i AND dp[end] is True

Code

```
def wordBreak(s, wordDict):
    root = TrieNode()
    for word in wordDict:
        cur = root
        for ch in word:
            if ch not in cur.children:
                cur.children[ch] = TrieNode()
            cur = cur.children[ch]
        cur.is_end = True

    n = len(s)
    dp = [False] * (n + 1)
    dp[n] = True
    for i in range(n - 1, -1, -1):
        cur = root
        for j in range(i, n):
            ch = s[j]
            if ch not in cur.children:
                break
            cur = cur.children[ch]
            if cur.is_end and dp[j + 1]:
                dp[i] = True
                break
    return dp[0]
```

start index	trie walk	dp effect
i=0	walk a-p-p-l-e	is_end at j=4, dp[5]=dp[9]? recurse
i=5	walk p-e-n	is_end at j=7, dp[8]=dp[13]=True
i=8	walk a-p-p-l-e	is_end at j=12, dp[13]=dp[13]=True(base)

Complexity

Let $n = \text{len}(s)$, $L = \text{max dictionary word length}$. The DP has n starting positions, and from each we walk the Trie up to L characters: $O(n \cdot L)$ time, $O(n)$ space for the `dp` array (plus the Trie's own space). Compare to a naive HashSet-based Word Break, which at each starting index tries every substring length and does a hash lookup per length — asymptotically similar, but the Trie version short-circuits the moment a prefix stops being valid at all, without needing to know the maximum dictionary word length in advance.

Extension 1 — Prefix AND Suffix Search

Recognition

LC 745. `WordFilter(words)` then `f(prefix, suffix)` returns the largest index of a word that has *both* the given prefix *and* the given suffix. A single ordinary Trie only answers prefix questions — it has no notion of "ends with".

Thought process

The trick: instead of indexing each word once, index **every (suffix, whole word) combination** as a single combined string `suffix + '#' + word`, all inserted into one shared Trie. `#` is a sentinel character that never appears in real words, so it cleanly separates "the suffix part" from "the whole word part" within one path.

A query `f(prefix, suffix)` then walks `suffix + '#' + prefix` through that same Trie. If that walk survives to the end, every word that could have produced this path is guaranteed to have **both** the queried prefix and the queried suffix — because the path only exists if some inserted `suffix' + '#' + word'` combination had `word'` starting with `prefix` *and* the indexed suffix starting with our query's `suffix`.

WordFilter: index "suffix#word" so ONE Trie answers both ends



word "care", suffix "e" -> inserted path "e" + "#" + "care"

query f(prefix="ca", suffix="e") walks "e#ca"

surviving node's stored weight = largest matching index

repeated for EVERY suffix of every word ($k = \text{len}(\text{word}) + 1$ insertions)

Each node additionally stores the **largest word index** that passed through it during insertion, so a successful query walk ends exactly at the answer.

Code

```
class WordFilterNode:
    __slots__ = ("children", "weight")

    def __init__(self):
        self.children = {}
        self.weight = -1
```

```

class WordFilter:
    """For every word, index every (suffix, whole-word) combo as the string
    `suffix + '#' + word`, inserted into one Trie. A query walks
    `suffix + '#' + prefix` through that same Trie: any surviving path is
    guaranteed to have both the right prefix and the right suffix, and each
    node remembers the largest index of a word that passed through it.
    """

    def __init__(self, words):
        self.root = WordFilterNode()
        for weight, word in enumerate(words):
            for k in range(len(word) + 1):
                combo = word[k:] + '#' + word
                cur = self.root
                cur.weight = weight
                for ch in combo:
                    if ch not in cur.children:
                        cur.children[ch] = WordFilterNode()
                    cur = cur.children[ch]
                cur.weight = weight

    def f(self, prefix, suffix):
        cur = self.root
        for ch in suffix + '#' + prefix:
            if ch not in cur.children:
                return -1
            cur = cur.children[ch]
        return cur.weight

```

Complexity

For n words with average length L : building the index inserts, for each word, one combined string per suffix — $O(n \cdot L^2)$ total insertions in the worst case (since a word of length L has $L+1$ suffixes, each up to $2L+1$ characters long once combined with `#word`). Each query is $O(\text{len(prefix)} + \text{len(suffix)})$. This trades index-build cost for very fast queries, which fits the problem's usage pattern (`WordFilter` is constructed once, `f()` is likely called many times).

Extension 2 — Design Search Autocomplete System

Recognition

LC 642. A stateful autocomplete: as the user types character by character, return the top-3 historical sentences matching what's been typed so far, ranked by how often each sentence has been used ("hot degree"), with ties broken lexicographically. `'#'` ends the current input and records it as a new (or incremented) sentence.

Thought process

This combines two things already covered in this guide: the **Trie + DFS autocomplete pattern** (Pattern 1) to find every sentence matching the characters typed so far, plus a **ranking rule** — sort DFS results by `(-frequency, sentence)` and take the top 3. The one new piece of state is a Trie node's `count`, incremented every time a sentence is typed to completion (either from the initial history or from the user finishing an input with `'#'`).

AutocompleteSystem: rank DFS results by (hot degree, then lexicographic)

i love you	count=5
island	count=3
iroman	count=2
i love leetcode	count=2

input('i') sorts matches by -count then string -> top 3 shown

Code

```
class ACNode:
    __slots__ = ("children", "is_end", "count")

    def __init__(self):
        self.children = {}
        self.is_end = False
        self.count = 0
```

```

class AutocompleteSystem:
    def __init__(self, sentences, times):
        self.root = ACNode()
        for s, t in zip(sentences, times):
            self._insert(s, t)
        self.cur = self.root
        self.buffer = []

    def _insert(self, sentence, delta):
        cur = self.root
        for ch in sentence:
            if ch not in cur.children:
                cur.children[ch] = ACNode()
            cur = cur.children[ch]
        cur.is_end = True
        cur.count += delta

    def _collect(self, node, prefix):
        results = []

        def dfs(n, path):
            if n.is_end:
                results.append((n.count, prefix + "".join(path)))
            for ch, child in n.children.items():
                path.append(ch)
                dfs(child, path)
                path.pop()

        dfs(node, [])
        # highest count first, then lexicographically smallest
        results.sort(key=lambda p: (-p[0], p[1]))
        return [text for _, text in results[:3]]

    def input(self, c):
        if c == '#':
            self._insert("".join(self.buffer), 1)
            self.buffer = []
            self.cur = self.root
            return []
        self.buffer.append(c)
        if self.cur is not None:
            self.cur = self.cur.children.get(c)
        if self.cur is None:
            return []
        return self._collect(self.cur, "".join(self.buffer))

```

Complexity

Let K = number of characters typed since the last '#', M = number of sentences matching the current prefix, L = average matching-sentence length. Each `input(c)` call does an $O(M \cdot L)$ DFS + sort to rank matches (this is the dominant cost; a heap of size 3 could shave the sort to $O(M \log 3)$ if optimizing further, but a full sort is simpler and fine for the interview-level solution). Recording a finished sentence is $O(\text{len}(\text{sentence}))$.

Extension 3 — Implement Trie II (Prefix Tree)

Recognition

LC 1804. A short recap problem that makes the "augment the node" technique from Map Sum Pairs completely explicit: track `countWordsEqualTo(word)`, `countWordsStartingWith(prefix)`, and support `erase(word)`.

Thought process

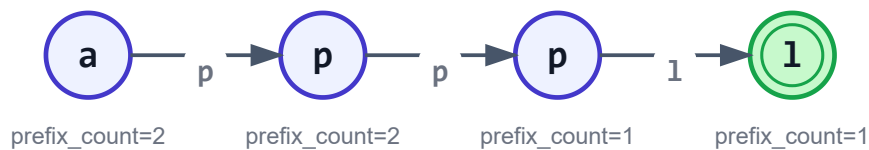
Instead of a boolean `is_end`, each node tracks two counters:

- `word_count` — how many times *this exact word* was inserted (a word can be inserted more than once).
- `prefix_count` — how many inserted words pass *through* this node at all (i.e. have this node's path as a prefix).

`insert` increments `prefix_count` on every node along the path and `word_count` on the final node.

`erase` walks the same path and **decrements** both — since `erase` is only ever called on a word known to exist, no existence check is needed beyond what the problem guarantees.

Trie II: every node also tracks `prefix_count` (path popularity)



`insert("appl")` twice, then `erase("appl")` once

`erase` walks the same path, decrementing `prefix_count` on every node

Code

```
class Trie2Node:
    __slots__ = ("children", "word_count", "prefix_count")

    def __init__(self):
        self.children = {}
        self.word_count = 0      # how many times THIS exact word was inserted
        self.prefix_count = 0   # how many inserted words pass through here
```

```

class Trie2:
    def __init__(self):
        self.root = Trie2Node()

    def insert(self, word):
        cur = self.root
        cur.prefix_count += 1
        for ch in word:
            if ch not in cur.children:
                cur.children[ch] = Trie2Node()
            cur = cur.children[ch]
            cur.prefix_count += 1
        cur.word_count += 1

    def _walk(self, word):
        cur = self.root
        for ch in word:
            if ch not in cur.children:
                return None
            cur = cur.children[ch]
        return cur

    def countWordsEqualTo(self, word):
        node = self._walk(word)
        return node.word_count if node else 0

    def countWordsStartingWith(self, prefix):
        node = self._walk(prefix)
        return node.prefix_count if node else 0

    def erase(self, word):
        cur = self.root
        path = [cur]
        for ch in word:
            cur = cur.children[ch]
            path.append(cur)
        path[-1].word_count -= 1
        for node in path:
            node.prefix_count -= 1

```

Complexity

Every operation (`insert` , `erase` , `countWordsEqualTo` , `countWordsStartingWith`) is **$O(L)$** where **L** is the word/prefix length — identical to the core Trie's complexity, just with richer per-node bookkeeping.

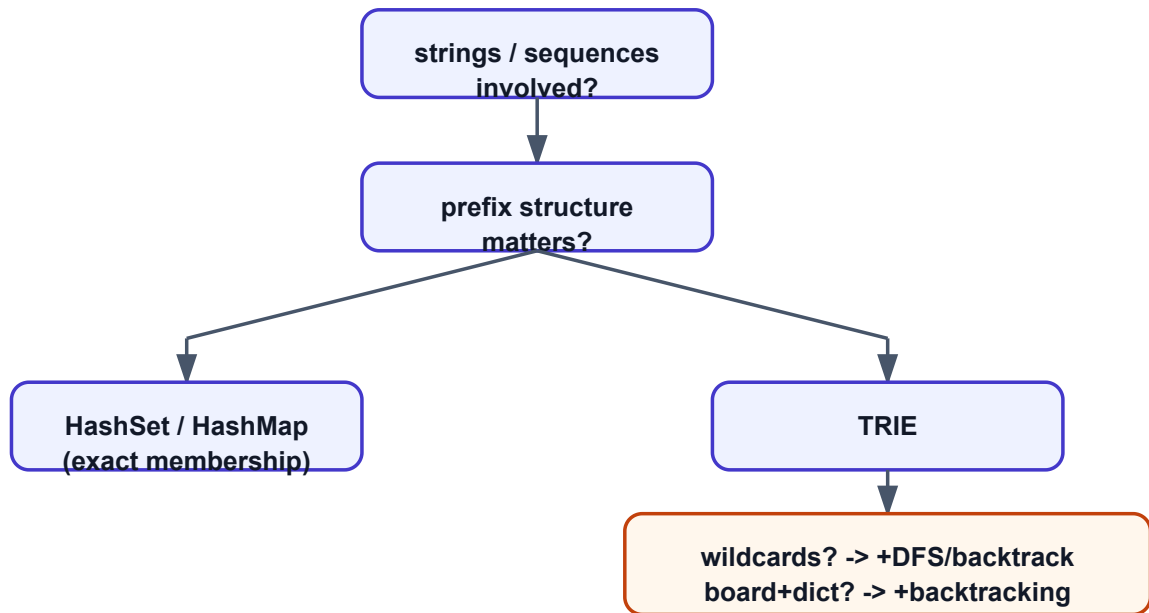
Recognition Guide

Ask these questions, in order, whenever a problem might be Trie-shaped:

1. **Are we dealing with sequences/strings (or bits)?** If no, a Trie is probably irrelevant.
2. **Does the problem care about prefixes?** Look for: "starts with", "common prefix", "autocomplete", "dictionary prefix", "possible continuation". If yes, consider a Trie.

3. **Do we repeatedly ask "can this partial sequence still become a valid stored sequence?"** → strong Trie signal (this is exactly what powers prefix pruning in Word Search II).
4. **Do we only need "does this exact word exist"?** → a HashSet/HashMap is probably simpler; don't reach for a Trie by default.
5. **Does the problem have wildcards** (`.` or similar "any character")? → Trie + DFS/backtracking.
6. **Backtracking through strings that needs prefix pruning?** → Trie + backtracking.

Trie recognition flowchart



The four Trie patterns you actually need

- | | |
|-----------------------------|--|
| 1. Trie traversal | insert / search / startsWith |
| 2. Trie + DFS | autocomplete, wildcards, dictionary collection |
| 3. Trie + backtracking | Word Search II – prune invalid prefixes |
| 4. Trie + another algorithm | Trie + DP (Word Break), binary Trie + greedy (Max XOR) |

Full decision table

Need	Natural choice	Why
Exact word membership	HashSet	Simple exact lookup
Key → value	HashMap	Maps keys to values
Prefix existence	Trie	Prefix is represented by a path
Autocomplete / prefix suggestions	Trie + DFS	Find the prefix node, then DFS
Wildcard dictionary search	Trie + DFS/backtracking	One character can mean multiple branches
Prefix pruning in backtracking (board + dictionary)	Trie + backtracking	Quickly reject invalid prefixes
Dictionary + string segmentation	Trie + DP	Trie finds word boundaries; DP combines them
Bitwise XOR optimization	Binary Trie + greedy	Bits are just another kind of "character"
Prefix AND suffix together	Combined suffix#prefix Trie	One Trie, indexed twice per word

Interview Q&A Drills

Q1. Store 1,000,000 usernames and repeatedly check whether a username exists exactly. Trie or HashSet? HashSet — this is pure exact membership; the prefix structure of a Trie buys nothing here and costs more memory.

Q2. Given 100,000 dictionary words, repeatedly ask whether "inter" is a prefix of any word. Trie or HashSet? Trie — `startsWith("inter")` is $O(\text{len}(\text{"inter"}))$ on a Trie regardless of dictionary size; a HashSet would need to scan all 100,000 words per query.

Q3. Given dictionary words and a board, find all dictionary words formable by walking adjacent cells. Trie, HashSet, or Trie + DFS? Trie + DFS/backtracking — the dictionary provides prefix structure to prune board DFS early (Word Search II).

Q4. Why is `search("app")` allowed to be `False` while `startsWith("app")` is `True`, given only `apple` and `application` were inserted? Because they ask different questions about the same node: `search` additionally requires `is_end == True` at the final node; `startsWith` only requires the path to exist. The node for "app" exists (both `apple` and `application` pass through it) but `is_end` was never set there, since "app" itself was never inserted as a complete word.

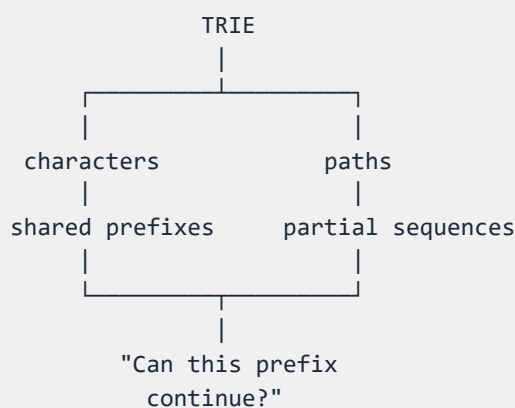
Q5. Why can inserting the same key twice into `MapSum` NOT simply add the new value to the old running totals? Because the old value's contribution is already baked into every ancestor's running total from the first insert. Adding the new value on top would double-count the original insert. The fix is to compute the **delta** between the new and old value and apply only that delta to every node on the key's path.

Q6. In Word Search II, why do we delete a Trie node's entry from its parent's children once that node has no children left? Because a dead-end node (no children, word already collected) can never contribute a valid match again from *any* future board cell — leaving it in the Trie only means later DFS calls waste time walking into a branch that can no longer produce a result.

Q7. What's the difference between the pruning in longestWord (LC 720) and the pruning in Word Search II (LC 212)? longestWord 's DFS is gated by is_end (only continue through nodes that are themselves complete stored words); Word Search II's DFS/ backtracking is gated by whether a **child exists at all** (continue only if the path could still lead to *some* dictionary word, complete or not). One prunes based on "is this itself a full word", the other prunes based on "could this possibly become a full word."

Q8. Why does a binary Trie for Maximum XOR insert numbers most-significant-bit first? Because the greedy query needs to make its most impactful decision first: the highest bit position contributes the most to the final XOR value, so choosing the opposite bit there (when available) locks in the largest possible gain before lower bits are considered — a greedy strategy only works correctly when the most valuable choices are made first.

Final mental model



HashSet asks: "Does this exact word exist?" **Trie asks:** "Where does this prefix take me, and what can continue from here?"

Once you recognize prefix structure, the implementation follows naturally: prefix structure → Trie → follow characters → children → is_end for complete words → DFS/backtracking if multiple continuations matter.